Отчет о классах и архитектуре проекта

Классы и архитектура программного комплекса для сравнения vision-LLM в задаче классификации самолётов

Отчет по назначению классов, объектной модели, математическим алгоритмам и взаимодействию компонентов (Практика №3)

В программном комплексе объявлено пять ключевых модулей, разделяющих ответственность за загрузку данных, кодирование изображений и нормализацию ответов, взаимодействие с облачными провайдерами, расчет метрик классификации и оркестрацию пайплайна. Конвейер обеспечивает как онлайн-инференс через Groq и Cloudflare Workers AI, так и детерминированный офлайн-анализ предварительно сохраненных предсказаний (`results/predictions.csv`).

1. Назначение и связь модулей

Модуль	Файл реализации	Ключевое назначение
Конфигурация	`config.py`	Централизованное хранение ключей, идентификаторов моделей, промпта, путей и параметров инференса (`RETRIES`, `REQUEST_DELAY`, `MAX_TOKENS`).
Загрузчик датасета	`dataset_loader.py`	Обход папок `dataset/civil` и `dataset/military`, фильтрация по `VALID_EXT`, возврат списка `{path, true_label}`, контроль

		баланса классов.
Утилиты	`utils.py`	Сжатие изображений (`PIL`, `thumbnail`, JPEG `quality=70`), кодирование в base64 `data`-URL и нормализация свободного ответа модели в метку {`civil`, `military`, `unknown`}.
Инференс	`inference.py`	Клиенты Groq (OpenAI-совместимый) и Cloudflare (`/ai/run`), ретраи, проверка доступности провайдеров, последовательный прогон датасета.
Метрики	`metrics.py`	Расчет accuracy, precision, recall, F1 по положительному классу `military`, матрица ошибок, консервативная обработка `unknown`, вывод ошибок.
Оркестратор	`main.py`	Сборка пайплайна: `load_dataset` → `run_all_models` → `predictions.csv` → `metrics.csv` → печать таблицы и ошибок.

2. Модуль конфигурации `config.py`

Модуль определяет глобальные константы, доступные всем остальным компонентам. Ключевое решение — вынести все изменяемые параметры в одно место, чтобы переключение модели или провайдера требовало правки ровно одной строки.

Поля и константы:

- `GROQ_API_KEY`, `CF_ACCOUNT_ID`, `CF_API_TOKEN` — учетные данные провайдеров. В финальной версии должны читаться из переменных окружения (`python-dotenv`), а не храниться в коде.
- `MODELS: dict[str, str]` — человекочитаемое имя модели (ключ) и ее идентификатор у провайдера (значение). Ключи используются как стабильные идентификаторы в `predictions.csv` и `metrics.csv`.
- `PROVIDERS: dict[str, str]` — соответствие «имя модели → провайдер» (значения `"groq"` или `"cloudflare"`). Позволяет `inference.py` выбирать нужную функцию вызова без `if-else` по имени модели.
- `PROMPT: str` — фиксированная инструкция, требующая ответа одним словом из закрытого множества `{'военный', 'гражданский'}`.
- `REQUEST_DELAY = 1.0`, `MAX_TOKENS = 16`, `TEMPERATURE = 0.0`, `RETRIES = 3` — параметры инференса, снижающие риск `429` и обеспечивающие детерминированность ответов.
- `LABELS = ["civil", "military"]`, `POSITIVE_LABEL = "military"` — контракт классов, используемый в `metrics.py` для `confusion matrix` и расчета `precision/recall/F1`.
- `RESULTS_DIR.mkdir(exist_ok=True)` — гарантирует существование папки для артефактов до первого запуска.

3. Загрузчик датасета `dataset_loader.py`

Модуль реализует единственную публичную функцию `load_dataset() -> list[dict]`.

Алгоритм:

1. Итерируется по `LABELS` (`["civil", "military"]`).
2. Для каждой метки открывает папку `DATASET_DIR / label`. Если папка отсутствует — печатает `[WARN]` и переходит к следующей метке (не падает).
3. Внутри папки перебирает файлы в отсортированном порядке, пропускает все, чье расширение не входит в `VALID_EXT` (`{.jpg, .jpeg, .png, .webp}`).
4. Для каждого подходящего файла добавляет словарь `{'path': str(img_path), 'true_label': label}`.
5. Печатает `[INFO]` с общим числом изображений и распределением по классам.

Возвращаемый контракт: список словарей, где `'path'` — строка (путь к файлу), `'true_label'` — одна из меток `LABELS`.

4. Утилиты `utils.py`

Модуль содержит две функции, разделяющие ответственность «подготовка входа» и «разбор выхода».

`encode_image_to_base64(image_path, max_size=512, quality=70)`

Открывает изображение через `PIL.Image.open`, принудительно конвертирует в `RGB` (это важно для PNG с альфа-каналом и для градаций серого), затем, если большая сторона превышает `max_size`, применяет `img.thumbnail((max_size, max_size), Image.LANCZOS)` — уменьшение с сохранением пропорций и высококачественной фильтрацией.

Сохраняет результат в `io.BytesIO` в формате JPEG с параметром `quality`, кодирует в `base64` и возвращает строку вида `"..."`.

Обоснование сжатия: стоимость запроса к vision-LLM пропорциональна числу визуальных патчей, то есть размеру изображения. Сжатие до 512 px и JPEG 70 снижает расход токенов на порядок без потери информативности для бинарной классификации.

`normalize_prediction(text) -> str`

Приводит текст к нижнему регистру и ищет ключевые слова двух семейств:

- `mil_keys = ['military', 'военн', 'боевой', 'истребитель', 'fighter', 'combat'];`
- `civ_keys = ['civil', 'гражданск', 'civilian', 'commercial', 'passenger', 'airliner', 'пассажирск'].`

Логика:

- найдено только одно семейство → соответствующая метка;
- найдены оба → берется то, чье первое вхождение раньше в строке (устойчиво к ответам вида `"civil, but looks military"`);
- ничего не найдено или пустой ответ → `"unknown"`.

Функция не бросает исключений и всегда возвращает одну из трех строк: `"civil"`, `"military"` или `"unknown"`.

5. Модуль инференса `inference.py`

Модуль инкапсулирует все обращения к внешним API. Ключевое архитектурное решение — две отдельные функции вызова (`call_groq` и `call_cloudflare`), потому что провайдеры используют разные форматы передачи изображения и разные схемы ответа.

5.1. Клиенты

`groq_client = Groq(api_key=GROQ_API_KEY)` — глобальный клиент Groq, создается один раз при импорте модуля.

5.2. `call_groq(model_id, image_b64) -> str`

Формирует `messages` в формате OpenAI: список из одного сообщения с `role="user"` и `content`, где `content` — список из двух элементов:

- `{'type': 'text', 'text': PROMPT}`;
- `{'type': 'image_url', 'image_url': {'url': image_b64}}`.

Вызывает `groq_client.chat.completions.create` с параметрами `model=model_id`, `max_tokens=MAX_TOKENS`, `temperature=TEMPERATURE`. Возвращает `response.choices[0].message.content` или пустую строку.

5.3. `call_cloudflare(model_id, image_path) -> str`

Читает файл в бинарном режиме и преобразует в `list(bytes)` (Cloudflare требует именно JSON-сериализуемый список байтов, а не base64).

Формирует URL `https://api.cloudflare.com/client/v4/accounts/{CF_ACCOUNT_ID}/ai/run/{model_id}` и POST-запрос с payload:

- `prompt = PROMPT`,
- `image = list(bytes)`,
- `max_tokens = MAX_TOKENS`,
- `temperature = TEMPERATURE`.

Заголовки: `Authorization: Bearer CF_API_TOKEN`, `Content-Type: application/json`. Таймаут 120 секунд.

Разбор ответа: `data.get("result", {})`; если `result` — `dict`, возвращается `result.get("response", "")`; иначе `str(result)`. Это защищает от двух возможных форматов, которые возвращает Cloudflare для разных моделей.

5.4. `call_model(model_id, image_b64, image_path, provider) -> str`

Универсальная обертка с ретраями. Для каждой попытки из `RETRIES`:

- выбирает `call_cloudflare` или `call_groq` по `provider`;
- при исключении печатает `[ERROR]` с номером попытки и спит `2 * (attempt + 1)` секунд (экспоненциальная пауза);
- после `RETRIES` неудач возвращает пустую строку (которая далее нормализуется в `"unknown"`).

5.5. `check_provider(provider, model_id) -> bool`

Легкий `ping`-запрос для проверки доступности модели:

- для Groq — `chat.completions.create` с `messages=[{"role":"user","content":"ping"}]`, `max_tokens=1`;
- для Cloudflare — POST на `/ai/run/{model_id}` с `{"prompt":"ping","max_tokens":1}`, проверка `status_code == 200`.

Любое исключение или не-200 → `False` с печатью `[FAIL]`.

5.6. `check_providers(models, providers) -> dict`

Проходит по всем моделям из `MODELS`, для каждой вызывает `check_provider` и собирает словарь только рабочих моделей. Печатает `[INFO]` с числом доступных моделей из общего числа.

5.7. `run_inference(dataset, model_name, model_id) -> list[dict]`

Прогоняет весь датасет через одну модель. Для каждого изображения:

- `encode_image_to_base64(item["path"])`;
- `call_model(...)` с провайдером из `PROVIDERS`;
- `normalize_prediction(raw)`;
- формирует запись `{model, image, true_label, raw_response, predicted_label}`, где `raw_response` обрезан до 120 символов и очищен от переводов строк;
- `time.sleep(REQUEST_DELAY)`.

5.8. `run_all_models(dataset, models) -> list[dict]`

Вызывает `check_providers`, затем последовательно прогоняет каждую рабочую модель через `run_inference`, накапливая записи в общий список. Порядок прогона определяется порядком ключей в `MODELS`.

6. Модуль метрик `metrics.py`

6.1. `compute_metrics(df) -> pd.DataFrame`

Для каждой модели (`groupby("model")`):

- `y_true = true_label`, `y_pred = predicted_label`;
- `n_total`, `n_unknown` — счетчики;
- `y_pred_fixed`: `unknown` заменяется на `POSITIVE_LABEL` (`"military"`), чтобы модель не получала незаслуженного преимущества за уклонение;
- `accuracy_score`, `precision_score`, `recall_score`, `f1_score` с `pos_label=POSITIVE_LABEL` и `zero_division=0` (защита от деления на ноль при вырожденных предсказаниях);

- `confusion_matrix(y_true, y_pred_fixed, labels=LABELS)` с безопасной распаковкой `tn, fp, fn, tp` (при неожиданном размере матрицы все четыре обнуляются);
- округление метрик до 3 знаков.

Возвращает DataFrame с колонками: `model`, `n_total`, `n_unknown`, `accuracy`, `precision_military`, `recall_military`, `f1_military`, `TP`, `TN`, `FP`, `FN`.

Сортировка — по `f1_military` по убыванию, чтобы лучшая модель была первой.

6.2. `print_errors(df) -> None`

Фильтрует строки, где `true_label != predicted_label`, и печатает для каждой: `[model] image: истина=..., предсказано=..., ответ='...'`. Если ошибок нет — печатает `"Ошибок нет."`.

7. Оркестратор `main.py`

Выполняет линейный пайплайн:

1. `dataset = load_dataset()` — если пусто, печатает предупреждение и завершает работу.
2. `all_records = run_all_models(dataset, MODELS)` — если пусто (ни одна модель не доступна), печатает предупреждение и завершает работу.
3. `df = pd.DataFrame(all_records); df.to_csv(PREDICTIONS_CSV, index=False, encoding="utf-8-sig")` — сохранение предсказаний с BOM для корректного открытия в Excel.
4. `metrics_df = compute_metrics(df); metrics_df.to_csv(METRICS_CSV, index=False, encoding="utf-8-sig")`.
5. `print(metrics_df.to_string(index=False))` — сравнительная таблица в консоль.

6. `print_errors(df)` — список ошибок.

8. Эмпирические результаты

На выборке из 26 изображений (13 `civil` + 13 `military`) получены следующие метрики:

Модель	Acc	Prec	Rec	F1	TP	TN	FP	FN
Llama-3.2-Vision (Cloudflare)	0.962	0.929	1.000	0.963	13	12	1	0
Qwen3.8-27B (Groq)	0.962	0.929	1.000	0.963	13	12	1	0

Обе модели показали идентичное качество. Единственная ошибка — изображение `c07.jpg` (истинный класс `civil`, предсказано `military`) — воспроизводится обеими моделями, что указывает на сложность самого изображения, а не на ограничения конкретной модели.

9. Заключение

Разработанный комплекс модулей полностью реализует требования учебного задания №3: обеспечивает воспроизводимый инференс двух vision-LLM через облачные API, устойчивую нормализацию ответов, корректный расчет метрик бинарной классификации с консервативной обработкой `unknown`, а также сохраняет все артефакты (`predictions.csv`, `metrics.csv`) для последующего офлайн-анализа и аудита.